

Trabajo Final Sistemas de Gestión de Bases de Datos:

"Arquitectura de Datos Híbrida para E-Commerce"

Objetivo: Desarrollar un sistema de gestión de datos para una tienda en línea, utilizando PostgreSQL para procesos transaccionales y MongoDB para información flexible del catálogo.

Instrucciones del Proyecto

Parte 1: Diseño y Estructura Relacional (PostgreSQL)

- i) Modelado DDL: Crear un script SQL que genere las tablas fundamentales: Clientes, Productos y Ventas.
- ii) Integridad de Datos: Implementar restricciones (Constraints) de tipo PRIMARY KEY, FOREIGN KEY (con comportamiento ON DELETE CASCADE), NOT NULL en campos críticos y al menos un CHECK para validar que los precios y el stock sean valores positivos .
- iii) Consulta de Negocio: Escribir una sentencia SQL que utilice un INNER JOIN para obtener un reporte que combine datos del cliente, el producto adquirido y el monto total de la venta .

Parte 2: Gestor Transaccional (Python + SQLAlchemy)

- i) Configuración del ORM: Crear un archivo Python que configure la conexión a PostgreSQL usando SQLAlchemy y mapee las tablas creadas en la Parte 1 a clases de Python.
- ii) Lógica de Venta: Programar una función que registre una venta y descuenta el stock del producto. Para asegurar la integridad de la base de datos, el bloque de código debe implementar un bloque try-except: debe confirmar los cambios con session.commit() si todo sale bien, o deshacerlos con session.rollback() si la operación falla, garantizando que el sistema nunca cobre un producto sin descontarlo del inventario.

Parte 3: Catálogo Dinámico (Python + MongoDB)

- i) Persistencia NoSQL: Conectar Python con MongoDB mediante pymongo.
- ii) Flexibilidad Documental: Crear una colección de Fichas_Tecnicas donde se inserten al menos dos productos con atributos diferentes (ej. un panel con "resistencia UV" y un accesorio con "dimensiones de tornillo"), demostrando la ventaja del esquema libre de NoSQL.
- iii) Búsqueda Avanzada: Realizar una consulta con filtros (find) para recuperar productos que cumplan con una especificación técnica particular.

Criterio	Excelente (25 pts)	Bueno (15 pts)	Necesita Mejora (5 pts)	Insuficiente (0 pt)
Modelado y Constraints (SQL)	Estructura perfecta con uso correcto de PK, FK, NOT NULL y CHECK. Datos íntegros.	Estructura funcional con PK y FK, pero omite restricciones de validación (CHECK/UNIQUE).	Errores en la definición de claves o tipos de datos mal asignados.	Ausencia de integridad referencial o script no funcional.
Consultas SQL (JOINS)	Implementa JOINS complejos que extraen información valiosa de múltiples tablas sin errores.	Logra unir tablas con JOIN, pero la consulta no responde con claridad a la lógica de negocio.	Presenta errores de sintaxis en el JOIN o los resultados son inconsistentes.	No logra relacionar tablas mediante SQL.
Lógica Transaccional (ORM)	Implementa el bloque try-except con commit y rollback de forma impecable. Garantiza atomicidad.	Usa el ORM para insertar y actualizar, pero la gestión de errores (rollback) es incompleta.	El código Python conecta con la DB pero no logra realizar la actualización de stock.	El código no utiliza transacciones o falla en la conexión inicial.
Integración NoSQL (MongoDB)	Demuestra uso real de la flexibilidad de documentos. Consultas con filtros funcionales.	Conecta e inserta en MongoDB, pero los documentos son rígidos y no aprovechan el esquema libre.	Errores en la manipulación de diccionarios o en el uso de la librería pymongo.	No implementa la capa de MongoDB o el código no es ejecutable.